ResearchGate

See discussions, stats, and author profiles for this publication at: https://www.researchgate.net/publication/315726273

A Survey of Itemset Mining

Article *in* Wiley Interdisciplinary Reviews: Data Mining and Knowledge Discovery · January 2017

CITATIONS
10

READS
143

6 authors, including:



Philippe Fournier Viger
Harbin Institute of Technology (Shenzhen)
213 PUBLICATIONS **1,858** CITATIONS
SEE PROFILE



Chun-Wei Jerry Lin
Høgskulen på Vestlandet
250 PUBLICATIONS **1,597** CITATIONS
SEE PROFILE



Bay Vo
Ho Chi Minh City University of Technology (HUTECH)
133 PUBLICATIONS **1,095** CITATIONS
SEE PROFILE



Tin C Truong
University of Dalat
22 PUBLICATIONS **138** CITATIONS
SEE PROFILE

Some of the authors of this publication are also working on these related projects:

Vision-based traffic surveillance View project

Novel algorithms for sequence mining, sequence prediction and utility pattern mining View project

All content following this page was uploaded by Philippe Fournier Viger on 02 December 2017.

The user has requested enhancement of the downloaded file.

Fournier-Viger, P., Lin, J. C.-W., Vo, B, Chi, T.T., Zhang, J., Le, H. B. (2017). **A Survey of Itemset Mining**. WIREs Interdisciplinary reviews - Data Mining and Knowledge Discovery, Wiley, to appear.

"This is the pre-peer reviewed version. This article may be used for non-commercial purposes in accordance with Wiley Terms and Conditions for Self-Archiving."

A Survey of Itemset Mining

Philippe Fournier-Viger [*], Jerry Chun-Wei Lin[†], Bay Vo[‡§], Tin Truong Chi[¶], Ji Zhang[‖], Hoai Bac Le [**]

Article Type:

Advanced Review

Abstract

Itemset mining is an important subfield of data mining, which consists of discovering interesting and useful patterns in transaction databases. The traditional task of frequent itemset mining is to discover groups of items (itemsets) that appear frequently together in transactions made by customers. Although itemset mining was designed for market basket analysis, it can be viewed more generally as the task of discovering groups of attribute values frequently co-occurring in databases. Due to its numerous applications in domains such as bioinformatics, text mining, product recommendation, e-learning, and web click stream analysis, itemset mining has become a popular research area.

This paper provides an up-to-date survey that can serve both as an introduction and as a guide to recent advances and opportunities in the field. The problem of frequent itemset mining and its applications are described. Moreover, main approaches and strategies to solve itemset mining problems are presented, as well as their characteristics. Limitations of traditional frequent itemset mining approaches are also highlighted, and extensions of the task of itemset mining are presented such as high-utility itemset mining, rare itemset mining, fuzzy itemset mining and uncertain itemset mining. The paper also discusses research opportunities and the relationship to other popular pattern mining problems such as sequential pattern mining, episode mining, sub-graph mining and association rule

[*]School of Natural Science and Humanities, Harbin Institute of Technology Shenzhen Graduate School, China

[†]School of Computer Science and Technology, University 2, Harbin Institute of Technology Shenzhen Graduate School, China

[‡]Faculty of Information Technology, Ho Chi Minh City University of Technology, Ho Chi Minh City, Vietnam

[§]College of Electronics and Information Engineering, Sejong University, Seoul, Republic of Korea

[¶]Department of Mathematics and Informatics, University of Dalat, Vietnam

[‖]Faculty of Health, Engineering and Sciences, University of Southern Queensland, Australia

[**]Faculty of Information Technology, University of Science, Vietnam

mining. Main open-source libraries of itemset mining implementations are also briefly presented.

INTRODUCTION

The goal of data mining is to predict the future or to understand the past[1,2]. Techniques used for predicting the future such as neural networks are often designed to behave as black-boxes because the goal is generally to obtain a model that is as accurate as possible rather than to obtain a model that is explanatory. On the other hand, several data mining techniques aim at discovering patterns in data that are understandable by humans. Approaches for discovering patterns in data can be classified by the types of patterns that they discover. Some common types of patterns found in databases are clusters, itemsets, trends, and outliers[2]. This paper is a survey that focuses on the discovery of itemsets in databases, a popular data mining task for analyzing symbolic data.

The task of discovering itemsets in databases was introduced in 1993 by Agrawal and Srikant[3] as *large itemset mining*, but it is nowadays called *frequent itemset mining* (FIM). The task of FIM is defined as follows. Given a database of customer transactions, FIM consists of discovering groups of items (*itemsets*) that are frequently purchased together by customers. For example, one may analyze a customer transaction database and discover that many customers buy taco shells with peppers. Discovering associations between items is useful to understand customer behavior. For instance, a retail store manager can use this knowledge to take strategic marketing decisions such as co-promoting products or putting them closer on the shelves.

Although FIM was originally proposed for analyzing customer data, it is now viewed as a general data mining task that is applicable to many domains. In fact, a customer transaction database can be more generally viewed as a database of instances describing objects (the transactions), where each object is described using nominal attribute values (the items). Thus, FIM can be equivalently defined as the task of finding attribute values that frequently co-occur in a database. Because many types of data can be represented as transaction databases, FIM has many applications in a wide-range of domains such as bioinformatics[5], image classification[6], network traffic analysis[7,8], analyzing customer reviews[9], activity monitoring[10], malware detection[11] and e-learning[12], to name just a few. FIM has also been extended in many ways to address specific needs. For example, some extensions of

TID	Transaction
T_1	$\{a, c, d\}$
T_2	$\{b, c, e\}$
T_3	$\{a, b, c, e\}$
T_4	$\{b, e\}$
T_5	$\{a, b, c, e\}$

Table 1: A transaction database

FIM are to discover rare patterns[13], correlated patterns[14–16], patterns in sequences[17,18] and graphs[19], and patterns that generate a high profit[20–26].

The field of itemset mining is a very active research field, where hundreds of new algorithms are proposed every year. This paper provides an up-to-date survey that can serve both as an introduction and as a guide to recent advances and opportunities in the field. The rest of this paper is organized as follows. The paper first describes the problem of FIM, and the main techniques employed in FIM. Then, the paper discusses popular extensions of the problem of FIM, and other problems in data mining that are closely related to FIM. Then, the paper discusses research opportunities and discusses open-source implementations for itemset mining. Finally, a conclusion is drawn.

Frequent itemset mining

The problem of frequent itemset mining is formally defined as follows[3]. Let there be a set of items (symbols) $I = \{i_1, i_2, \dots i_m\}$. A transaction database $D = \{T_1, T_2 \dots T_n\}$ is a set of transactions such that each transaction $T_q \subseteq I (1 \leq q \leq m)$ is a set of distinct items, and each transaction T_q has a unique identifier q called its TID (Transaction IDentifier). For example, consider the transaction database shown in Table 1. This database contains five transactions, where the letters a, b, c, d, e represents items bought by customers. For example, the first transaction T_1 represents a customer that has bought the item a, c and d.

An *itemset* X is a set of items such that $X \subseteq I$. Let the notation $|X|$ denote the set cardinality or, in other words, the number of items in an itemset X. An itemset X is said

to be of length k or a k-itemset if it contains k items ($|X| = k$). The goal of itemset mining is to discover interesting itemsets in a transaction database, that is interesting associations between items. In general, in itemset mining, various measures can be used to assess the interestingness of patterns. In FIM, the interestingness of a given itemset is traditionally defined using a measure called the *support*. The support (or *absolute support*) of an itemset X in a database D is denoted as $sup(X)$ and defined as the number of transactions containing X, that is $sup(X) = |\{T|X \subseteq T \wedge T \in D\}|$. For example, the support of the itemset $\{a, b\}$ is 2 because this itemset appears in two transactions (T_3 and T_5). Note that some authors prefer to define the support of an itemset X as a ratio. This definition called the *relative support* is $relSup(X) = sup(X)/|D|$. For example, the relative support of the itemset $\{a, b\}$ is 0.4.

The *task of frequent itemset mining*[3] consists of discovering all *frequent itemsets* in a given transaction database. An itemset X is *frequent* if it has a support that is no less than a given minimum support threshold *minsup* set by the user (i.e. $sup(X) \geq minsup$). For example, if we consider the database shown in Table 1 and that the user has set $minsup = 3$, the task of FIM is to discover all groups of items appearing in at least three transactions. In this case, there are exactly nine frequent itemsets: $\{a\} : 3$, $\{b\} : 4$, $\{c\} : 4$, $\{e\} : 4$, $\{a, c\} : 3$, $\{b, c\} : 3$, $\{b, e\} : 4$, $\{c, e\} : 3$, $\{b, c, e\} : 3$, where the number besides each itemset indicates its support.

FIM is an enumeration problem. The goal is to enumerate all patterns that meet the minimum support constraint specified by the user. Thus, there is always a single correct answer to a FIM task. FIM is a difficult problem. The naive approach to solve this problem is to consider all possible itemsets to then output only those meeting the minimum support constraint specified by the user. However, such a naive approach is inefficient for the following reason. If there are m distinct items in a transaction database, there are $2^m - 1$ possible itemsets. If a database has for example 1000 distinct items, the number of possible itemsets is $2^{1000} - 1$ itemsets, which is clearly unmanageable using a naive approach. It is important to note that the FIM problem can be very difficult even for a small database. For example, a database containing a single transaction with 100 items, with $minsup = 1$ still generates a search space of 2^{100} itemsets. Thus, the number of itemsets in the search space generally

matters more than the size of the data in FIM. But what influences the number of itemsets in the search space? The number of itemsets depends on how similar the transactions are in the database, and also on how low the *minsup* threshold is set by the user.

To discover frequent itemsets efficiently, it is thus necessary to design algorithms that avoid exploring the search space of all possible itemsets and that process each itemset in the search space as efficiently as possible. Several efficient algorithms have been proposed for FIM. Some of the most famous are Apriori[3], FP-Growth[27], Eclat[28], H-Mine[29] and LCM[30]. All of these algorithms have the same input and the same output. However, the difference is the strategies and data structures that these algorithms employ to discover frequent itemsets efficiently. More specifically, FIM algorithms differ in (1) whether they use a depth-first or breadth-first search, (2) the type of database representation that they use internally or externally, (3) how they generate or determine the next itemsets to be explored in the search space, and (4) how they count the support of itemsets to determine if they satisfy the minimum support constraint.

In the rest of this section, we first explain the concept of breadth-first search and depth-first search in itemset mining. Then, we provide an overview of the key techniques used by some of the most popular FIM algorithms and explain their advantages and limitations. These techniques are very important as they have inspired numerous algorithms in the field of pattern mining.

Breadth-first search and depth-first search

Most of the existing itemset mining algorithms can be described as either using a breadth-first search or a depth-first search. Assume that there are m items in a database. A breadth-first search algorithm (also called a level-wise algorithm) such as Apriori explores the search space of itemsets by first considering 1-itemsets, then 2-itemsets, 3-itemsets ..., and lastly m-itemsets. For example, Fig. 1 depicts the search space of all possible itemsets for the running example. In this figure, the search space is represented as a Hasse diagram[1]. A breadth-first search algorithm will first consider 1-itemsets $\{a\}, \{b\}, \{c\}, \{d\}$ and $\{e\}$. Then,

[1]A Hasse diagram draws an arrow from an itemset X to another itemset Y if and only if $X \subseteq Y$ and $|X| + 1 = |Y|$

it will generate 2-itemsets such as $\{a, b\}, \{a, c\}, \{a, d\}$, and then 3-itemsets, and so on, until it generates the itemset $\{a, b, c, d, e\}$ containing all items. On the other hand, depth-first search algorithms such as FPGrowth, H-Mine and LCM start from each 1-itemset and then recursively try to append items to the current itemset to generate larger itemsets. For example, in the running example, a typical depth-first search algorithm would explore itemsets in that order: $\{a\}, \{a, b\}, \{a, b, c\}, \{a, b, c, d\}, \{a, b, c, d, e\}, \{a, b, c, e\}$, $\{a, b, d\}$, $\{a, b, d, e\}$, $\{a, b, e\}$, $\{a, c\}$, $\{a, c, d\}$, $\{a, c, d, e\}$, $\{a, c, e\}$, $\{a, d\}$, $\{a, d, e\}$, $\{a, e\}$, $\{b\}$, $\{b, c\}$, $\{b, c, d\}$, $\{b, c, d, e\}$, $\{b, c, e\}$, $\{b, d\}$, $\{b, d, e\}$, $\{b, e\}$, $\{c\}$, $\{c, d\}$, $\{c, d, e\}$, $\{c, e\}$, $\{d\}$, $\{d, e\}$, $\{e\}$.

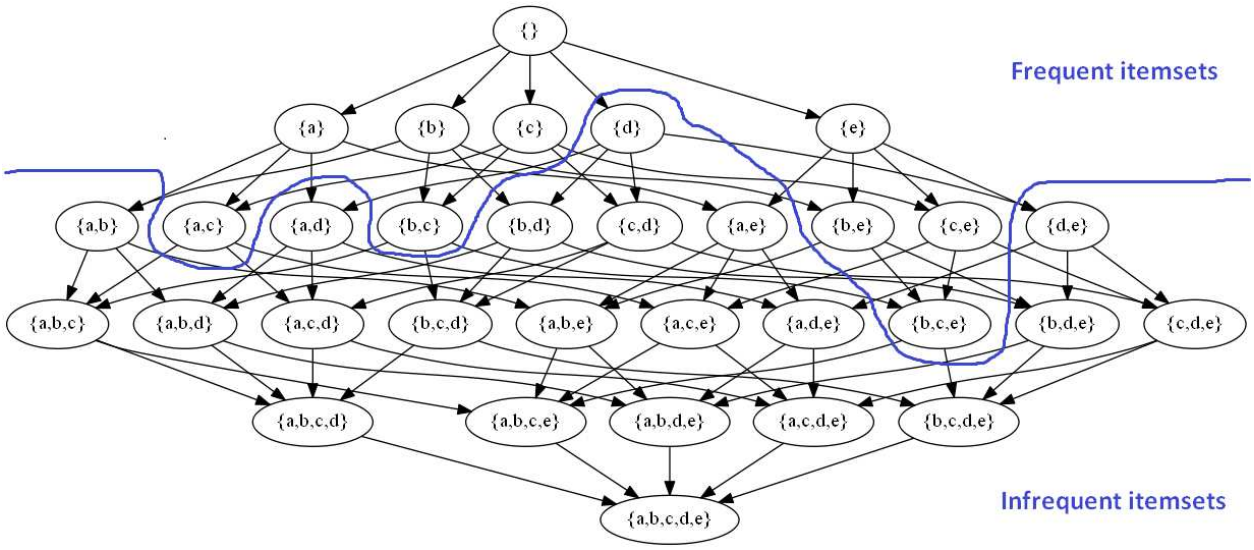


Figure 1: The search space for $I = \{a, b, c, d, e\}$.

To design an efficient FIM algorithm, it is important that the algorithm avoid exploring the whole search space of itemsets because the search space can be very large. To reduce the search space, *search space pruning techniques* are used. In FIM, the key observation for reducing the search space is that the support is a monotone measure, that is for any itemsets X and Y such that $X \subset Y$, it follows that $sup(X) \geq sup(Y)$[3]. This has the implication that if an itemset is infrequent, all its supersets are also infrequent, and thus do not need to be explored. For example, assuming that $minsup = 3$ and that the support of the itemset $\{a, b\}$ is 2, it can be concluded that $\{a, b\}$ and all its supersets are infrequent and do not need to be considered. This property, called the *downward-closure property*[3], *anti-monotonicity-property* or *Apriori-property* is very powerful and can greatly reduce the search space. For

example, it can be observed in Fig. 1 that only 9 itemsets are frequent out of 31 possible itemsets (excluding the empty set). Thus by applying the above property, many itemset can potentially be avoided when exploring the search space.

Apriori: an horizontal breadth-first search algorithm

Apriori[3] is the first FIM algorithm. Apriori takes a transaction database and the *minsup* threshold as input. Apriori uses a standard database representation, as shown in Table 1, also called a *horizontal database*. The pseudocode of Apriori is given in Algorithm 1. Apriori first scans the database to calculate the support of each item, i.e. 1-itemset (line 1). Then, Apriori uses this information to identify the set of all frequent items, denoted as F_1 (line 2). Then, Apriori performs a breadth-first search to find larger frequent itemsets (line 4 to 10). During the search, Apriori uses the frequent itemsets of a given length $k-1$ (denoted as F_{k-1}) to generate potentially frequent itemsets of length k (denoted as C_k). This is done by combining pairs of items of length k that share all but one item (line 5). For example, if the frequent 1-itemsets are $\{a\}, \{b\}, \{c\}$ and $\{e\}$, Apriori will combine pairs of these itemsets to obtain the following candidate 2-itemsets: $\{a, b\}$, $\{a, c\}$, $\{a, e\}$, $\{b, c\}$, $\{b, e\}$, and $\{c, e\}$. After generating candidates of length k, Apriori checks if the $(k-1)$-subsets of each candidate are frequent. If a candidate itemset X has an infrequent $(k-1)$-subset, X cannot be frequent (it would violate the downward-closure property) and it is thus removed from the set of candidate k-itemsets. Then, Apriori scans the database to calculate the support of all remaining candidate itemsets in C_k (line 7). Each candidate having a support not less than *minsup* is added to the set F_k of frequent k-itemsets (line 8). This process is repeated until no candidates can be generated. The set of all frequent itemsets is then returned to the user (line 11).

Apriori is an important algorithm as it has inspired many other algorithms. However, it suffers from important limitations. The first one is that because Apriori generates candidates by combining itemsets without looking at the database, it can generate some patterns that do not even appear in the database. Thus, it can spend a huge amount of time processing candidates that do not exist in the database. The second limitation is that Apriori has to repeatedly scan the database to count the support of candidates, which is very costly. The

third limitation is that the breadth-first search approach can be quite costly in terms of memory as it requires at any moment to keep in the worst case all k and $k-1$ itemsets in memory (for $k > 1$). In terms of complexity, a very detailed complexity analysis of the Apriori algorithm has been done by Hegland[31]. Briefly, the time complexity is $O(m^2 n)$, where m is the number of distinct items and n is the number of transactions.

Algorithm 1: The Apriori algorithm

input : D: a horizontal transaction database, *minsup*: a user-specified threshold

output: the set of frequent itemsets

1 Scan the database to calculate the support of all items in I;
2 $F_1 = \{i | i \in I \wedge sup(\{i\}) \geq minsup\}$; // F_1 : frequent 1-itemsets
3 $k = 2$;
4 **while** $F_k \neq \emptyset$ **do**
5 $C_k =$ CandidateGeneration (F_{k-1}) ; // C_k : candidate k-itemsets
6 Remove each candidate $X \in C_k$ that contains a $(k-1)$-itemset that is not in F_{k-1};
7 Scan the database to calculate the support of each candidate $X \in C_k$;
8 $F_k = \{X | X \in C_k \wedge sup(X) \geq minsup\}$; // F_k : frequent k-itemsets
9 $k = k + 1$;
10 **end**
11 return $\bigcup_{k=1...k} F_k$;

Eclat: a vertical depth-first search algorithm

The Eclat[28] algorithm improves upon the Apriori approach by using a depth-first search to avoid keeping many itemsets in memory. Contrarily to Apriori, Eclat utilizes what is called a *vertical database representation*. A vertical database representation indicates the list of transactions where each item appears. For an itemset i, the list of transactions containing the item i is called its TID-list, and it is denoted as $tid(X)$. For example, the vertical representation of the database presented in Table 1 is shown in Table 2.

This vertical representation can be obtained by scanning the original horizontal database

Item (x)	**TID-set** ($tid(x)$)
a	$\{T_1, T_3, T_5\}$
b	$\{T_2, T_3, T_4, T_5\}$
c	$\{T_1, T_2, T_3, T_5\}$
d	$\{T_1\}$
e	$\{T_2, T_3, T_4, T_5\}$

Table 2: The vertical representation of the database of Table 1

only once. Furthermore, note that it is also possible to regenerate a horizontal database from a vertical database. The vertical representation is very useful in itemset mining because it possesses the following two properties. First, for any itemsets X and Y, the TID-list of the itemset $X \cup Y$ can be obtained without scanning the original database by intersecting the TID-lists of X and Y, that is: $tid(X \cup Y) = tid(X) \cap tid(Y)$. Second, the TID-list of an itemset X allows to directly derive its support without scanning the database, by using the property that $sup(X) = |tid(X)|$. For example, the TID-list of $\{a, c\}$ can be calculated as $tid(\{a, c\}) = tid(a) \cap tid(c) = \{T_1, T_3, T_5\}$, and it can thus be derived that the support of $\{a, c\}$ is $|tid(\{a, c\})| = 3$. Thus, using these two properties, *vertical algorithms* such as Eclat can explore the search space by scanning the database only once to create the initial TID-lists. Candidate generation and support counting is done directly without scanning the database.

The pseudocode of Eclat is shown in Algorithm 2. The Eclat algorithm takes as input a vertical database (a set R containing all items annotated with their tidsets, as shown in Table 2) and the *minsup* threshold. The Eclat algorithm performs a loop where it considers each itemset X in R that is frequent according to its tidset (line 2 to 10). The itemset X is first output. Then, a search is performed to find frequent itemsets extending X with one item. This is done by attempting to combine each itemset Y in R that shares all but the last item with X to obtain an itemset $X \cup Y$ (line 4 to 10). For example, if $X = \{a\}$, Eclat will attempt to combine X with $\{b\}, \{c\}, \{d\}$ and $\{e\}$ to generate the extensions $\{a, b\}$, $\{a, c\}$, $\{a, d\}$ and $\{a, e\}$, respectively. During this process, the tidset of an extension $X \cup Y$

is calculated as $tid(X) \cap tid(Y)$ (line 6). Then, if $X \cup Y$ is frequent according to its tidset, $X \cup Y$ is added to a set E of frequent extensions of X (line 6 and 7). After that the Eclat algorithm is recursively called with E to explore all extensions of $X \cup Y$. Then, the loop of line 1 to 10 is repeated for other itemsets in R. When the algorithm terminates, all frequent itemsets have been output.

The Eclat algorithm is considered to be a depth-first search algorithm as it outputs all frequent itemsets according to the depth-first search order. Eclat is generally much faster than Apriori as it does not perform multiple database scans. However, Eclat still has some drawbacks. First, because Eclat also generates candidates without scanning the database, it can spend time considering itemsets that do not exist in the database. Second, although TID-lists are useful, they can consume a lot of memory especially for dense datasets (datasets where all items appear in almost all transactions). It is important to note however, that there has been some work to reduce the size of TID-lists using an improved structure called diffsets[32]. Also other proposed improvements are for example to encode TID-lists as bit vectors[32,33] to reduce the memory usage and the speed of intersecting TID-lists, on dense datasets. TID-lists can also be used in breadth-first search algorithms. For example, Apriori-TID is a version of Apriori that relies on TID-lists to calculate the support of itemsets[3]

Pattern-growth algorithms

To address the main limitation of algorithms such as Apriori and Eclat, a major advance in the field has been the development of *pattern-growth algorithms* such as FP-Growth[27], H-Mine[29] and LCM[30]. The main idea of pattern-growth algorithms is to scan a database to find itemsets, and thus avoid generating candidates that do not appear in the database. Furthermore, to reduce the cost of scanning the database, pattern-growth algorithms have introduced the concept of *projected database* to reduce the size of databases as an algorithm explore larger itemsets with its depth-first search.

The pseudocode of a typical pattern-growth algorithm is shown in Algorithm 3. It takes as input a transaction database D, the empty set, and the *minsup* threshold. Without loss of generality, assume that there exists a total order on items $\prec$ such as the lexicographical order ($a \prec b \prec c \prec d \prec e$). A pattern-growth algorithm explores the search space using

Algorithm 2: The Eclat algorithm

input : R: a set of itemsets with their tidsets, $minsup$: a user-specified threshold

output: the set of frequent itemsets

```
1  foreach itemset X ∈ R such that |tid(X)| ≥ minsup do
2      Output X;                                          // X is a frequent itemset
3      E = ∅;                                  // frequent itemsets that are extensions of X
4      foreach itemset Y ∈ R sharing all but the last item with X do
5          tid(X ∪ Y) = tid(X) ∩ tid(Y);          // calculate the tidset of X ∪ Y
6          if |tid(X ∪ Y)| ≥ minsup then                  // if X ∪ Y is frequent
7              E = E ∪ {X ∪ Y};          // add X ∪ Y to frequent extensions of X
8          end
9          Eclat (E, minsup);                             // recursive call using E
10     end
11 end
```

a depth-first search by recursively appending items according to the $\prec$ order to frequent itemsets, to obtain larger frequent itemsets. At the beginning, a pattern-growth algorithm considers that the current itemset X is the empty set. A pattern-growth algorithm scans the database D to find the set Z of all frequent items in D (line 1). Then, for each such item z, the itemset $X \cup \{z\}$ is output as it is a frequent itemset (line 3). Then, the pattern-growth procedure is called to perform a depth-first search to find larger frequent itemsets that are extensions of $X \cup \{z\}$ in the same way (line 5). However, it can be observed that not all items in D can be appended to $X \cup \{z\}$ to generate larger itemsets. In fact, the itemset $X \cup \{z\}$ may not even appear in all transactions of the database D. For this reason, a pattern-growth algorithm will create the projected database of the itemset $X \cup \{z\}$ (line 4) and will use this database to perform the depth-first search (line 5). This will allows reducing the cost of scanning the database. After recursively performing the depth-first search for all items, the set of all frequent itemsets will have been output.

Now, let's illustrate these steps in more details with an example. Assume that $minsup = 3$. By scanning the database of Table 1, it can be found that the frequent 1-itemsets are a, b, c

and e. The algorithm will first consider the item a to try to find larger frequent itemsets starting with the prefix $\{a\}$. The algorithm will thus build the projected database of $\{a\}$ as shown in Table 3. The projected database of an item i is defined as the set of transactions where i appears, but where the item i and items preceding i according to the $\prec$ order have been removed. Then, to find frequent itemsets starting with $\{a\}$ containing one more item, the algorithm will scan the projected database of $\{a\}$ and count the support of all items appearing in that database. For example, the support of items in the projected database of $\{a\}$ are: $\{b\} : 2$, $\{c\} : 3$ and $\{e\} : 1$. This means that the support of $\{a, b\}$ is 2, that the support of $\{a, c\}$ is 3, and that the support of $\{a, e\}$ is 1. Thus, only the itemset $\{a, c\}$ is frequent (recall that we assume that $minsup = 3$ in the running example). Then the algorithm will pursue the depth-first search to find frequent itemsets starting with the prefix $\{a, c\}$. The algorithm will build the projected database of $\{a, c\}$ from the projected database of $\{a\}$. The projected database of $\{a, c\}$ is shown in Table 4. Then, the algorithm will scan the projected database of $\{a, c\}$ to find frequent items in that database. This process will continue until all frequent itemsets have been explored by the depth-first search.

A major advantage of pattern-growth algorithms is that they only explore the frequent itemsets in the search space thus avoiding considering many itemsets not appearing in the database, or infrequent itemsets. Besides, the concept of projected database is also useful to reduce the cost of database scans. A common question about the concept of projected database is: is it costly to create all these copies of the original database? The answer is no if an optimization called *pseudo-projection* is used, which consists of implementing a projected database as a set of pointers on the original database[29,30]. For example, Fig. 2 shows the pseudo-projected database of $\{a, c\}$, which is equivalent to the projected database of Table 4, excepts that it is implemented using three pointers on the original database, to avoid creating a copy of the original database. Note that many other optimizations can also be integrated in pattern-growth algorithms. For example, LCM[30] also integrates a mechanism to merge identical transactions in projected databases to further reduce their size, and an efficient array-based support counting technique called *occurrence-delivery*. The FP-Growth[27] and H-Mine[29] algorithms respectively introduce a prefix-tree structure and a hyper-structure for representing projected-databases to also reduce memory usage.

Algorithm 3: A pattern-growth algorithm

input : D: a transaction database, X: the current itemset (initially $X = \emptyset$),
minsup: a user-specified threshold

output: the set of frequent itemsets

```
1 Scan the database D to find the set Z of all frequent items in D;
2 foreach item z ∈ Z do
3     Output X ∪ {z};                          // X ∪ {z} is a frequent itemset
4     D' = Projection(D, z) ;                  // Create projected database of X ∪ {z}
5     PatternGrowth (D, X ∪ {z}, minsup);      // recursive call to extend X ∪ {z}
6 end
```

TID	Transaction
T_1	$\{c, d\}$
T_3	$\{b, c, e\}$
T_5	$\{b, c, e\}$

Table 3: The projected database of $\{a\}$

TID	Transaction
T_1	$\{d\}$
T_3	$\{e\}$
T_5	$\{e\}$

Table 4: The projected database of $\{a, c\}$

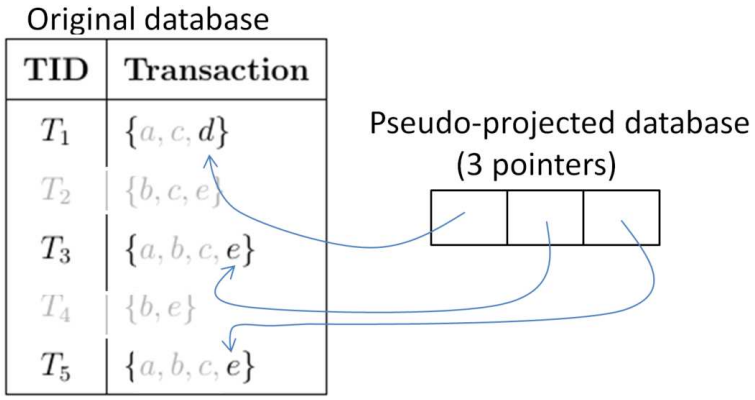


Figure 2: The pseudo-projected database of {a,c}

Table 5 provides a summary of the characteristics of the algorithms discussed in this section. In recent years, a lot of research has been carried on further improving the performance of algorithms for FIM since it is a computationally expensive task. These improvements have

Algorithm	Type of search	Database representation
Apriori [3]	breadth-first (candidate generation)	Horizontal
$Apriori-TID$ [3]	breadth-first (candidate generation)	Vertical (TID-lists)
Eclat[28]	depth-first (candidate generation)	Vertical (TID-lists, diffsets)
$FP-Growth$[27]	depth-first (pattern-growth)	Horizontal (prefix-tree)
$H-Mine$[29]	depth-first (pattern-growth)	Horizontal (hyperlink structure)
LCM[30]	depth-first (pattern-growth)	Horizontal (with transaction merging)

Table 5: A summary of six representative FIM algorithms

been in terms of proposing novel algorithms with additional optimizations[34–37], and also to design FIM algorithms that can run on GPU processors[38], on multi-core processors[39], and cloud platforms such as Hadoop[40] and Spark[41].

Variations of the Itemset Mining Problem

Although the task of frequent itemset mining has numerous applications, it can also be viewed as having limitations in terms of the assumptions that it makes. This section reviews some of the most important limitations of FIM and discusses extensions to the task of itemset mining that are designed to address these limitations.

One the most important limitation of FIM is that an algorithm may find a huge amount of itemsets, depending on how the minimum support threshold is set. Discovering too many patterns makes it difficult for a human to analyze the patterns found. Moreover, itemsets can be viewed as containing a lot of redundancy, since, for example, if an itemset is frequent, all its subsets are also frequent. To reduce the number of itemsets found and present more meaningful itemsets to the user, researchers have designed algorithms to extract *concise representations* of frequent itemsets. A concise representation is a set of frequent itemsets that is smaller and summarize the whole set of all frequent itemsets. In practice the size of concise representations can sometimes be several orders of magnitude smaller than the set of frequent itemsets. Moreover, discovering these concise representations is often faster than discovering the full set of frequent itemsets[30,33,42–48]. It was also shown that for task such as

classification, using concise representations of items can increase classification accuracy[50]. Let FI denotes the set of frequent itemsets. The most popular concise representations of frequent itemsets are the following.

- *Closed itemsets*[30,33,42–45] are the frequent itemsets that have no superset having the same support, i.e. $CI = \{X | X \in FI \wedge \nexists Y \in FI$ such that $X \subset Y \wedge sup(X) = sup(Y)\}$. In the example of Table 1, out of nine frequent itemsets, only four of them are closed: $\{c\}$, $\{e\}$, $\{a, c\}$, $\{b, c, e\}$. Thus, a considerable reduction in terms of number of itemsets is achieved by discovering only closed itemsets. An interesting property of closed itemsets is that they are a *lossless* representation of all frequent itemsets. In other words, using closed itemsets the information about all frequent itemsets, including their support, can be recovered without scanning the database[30,33,42–45]. In market basket analysis, closed itemsets are also interesting because they represent the largest sets of items common to groups of customers.

- *Maximal itemsets*[30] are the set of frequent itemsets that do not have supersets that are frequent, i.e. $MI = \{X | X \in FI \wedge \nexists X \in FI$ such that $X \subset Y\}$. In other words, maximal itemsets are the largest frequent itemsets. The representation of maximal itemsets is a subset of the representation of closed itemsets ($MI \subseteq CI \subseteq FI$), and thus can further reduce the number of itemsets presented to the user. However, maximal itemsets are not a lossless representation. Maximal itemsets can be used to recover all frequent itemsets, but they cannot be used to recover the support of all frequent itemsets. In the example of Table 1, there are only two maximal itemsets: $\{a, c\}$ and $\{b, c, e\}$.

- *Generator itemsets (key itemsets)*[46–48] are the set of frequent itemsets that have no subsets having the same support, i.e. $GI = \{X | X \in FI \wedge \nexists X \in FI$ such that $Y \subset X \wedge sup(X) = sup(Y)\}$. The set of generator itemsets is always of equal size as or larger than the set of closed itemsets and the set of maximal itemsets. But the set of generators is interesting according to the Minimum Description Length principle[49] since it represents the smallest sets of items that are common to sets of transactions. For example, in market basket analysis, generator itemsets represent the smallest sets

of items common to group of customers.

Another popular way of reducing the number of patterns found by itemset mining algorithms is to introduce constraints to filter less interesting patterns. Many different types of more or less complex constraints have been introduced. The naive way of applying constraints in FIM is to apply them as a post-processing step after all the frequent itemsets have been found. However, this approach may suffer from important performance issues since discovering all patterns can be time-consuming and require a lot of space. A better way of applying constraints is to push them deep in the mining process, i.e. to use the constraints to reduce the search space, and thus improve the performance of the algorithms. Algorithms that use constraints to reduce the search space can sometimes be orders of magnitude faster, since constraints can greatly reduce the size of the search space, and produce orders of magnitude less patterns. Some of the first itemset mining algorithms to integrate constraints are the Reorder and Direct algorithms, which are Apriori-based algorithms that let the user specify boolean constraints on items that can appear in frequent itemsets[51]. Another example of constraint is the use of the *occupancy*[52], a measure used to find itemsets that occupy a large portion of transactions where they appear. Measures have also been presented to assess how correlated each itemset is[14–16]. For example, the *bond*[14] of an itemset $X = \{i_1, i_2, \ldots i_p\}$ is defined as $bond(X) = |tid(X)|/|tid(i_1) \cap tid(i_2) \cap \ldots tid(i_p)|$. Thus, a frequent itemset having a high bond is not just frequent but also contains items that often co-occur. Many other alternative interestingness measures have also been proposed in the itemset mining literature[53] such as the affinity[55], all-confidence[54], coherence and mean[15,16], each having different advantages and limitations. The interested reader may refer to the paper of Geng et al.[53] for more details.

In terms of constraints that can be used in itemset mining, not all constraints can be pushed deep into the mining process to reduce the search space. Some studies[56–58] have categorized constraints into several categories such as monotone, anti-monotone, succinct, convertible and other constraints. *Anti-monotone constraints* such as the minimum support threshold are some of the easiest and most beneficial to integrate in a FIM mining algorithm, as they can be used to prune the search space by applying the downward closure property. *Convertible constraints* are constraints that are neither monotone nor anti-monotone but

that can be converted to anti-monotone constraints if some additional strategies are applied by the FIM algorithm. For example, assume that a weight is assigned to each item. A constraint that is neither anti-monotone nor monotone is the maximum average weight of an itemset. But this constraint can be easily converted into an anti-monotone constraint if items are sorted by increasing order of weights, and itemsets are explored by following that order [56]. A *succinct constraint* is a constraint that can be checked for an itemset by only looking at the single items that it contains. For example, the constraint that the sum of the weights of an itemset should be less than a given value can be checked by simply adding the weights of its items. This constraint is both succint and anti-monotone. For more information about the use of constraints, the reader may refer to the referenced papers[56–58] .

Another limitation of traditional FIM is that it assumes that all items are equal. But in real-life applications, items are often very different from each other[59]. For example, two items such as bread and caviar do not have the same selling frequencies in a retail store, and one should not expect these items to have the same frequencies since bread is a very common type of product while caviar is a specialized and expensive product. Thus, some items have naturally more chance of being frequent than others. This leads to the *rare item problem*[59], which means that some items are much less likely to appear in frequent itemsets than others. To address this issue, researchers have developed algorithms to find frequent itemsets using multiple minimum support thresholds, such as MSApriori[59], CFPGrowth[60] and CFPGrowth++[61]. These algorithms let the user select a different minimum support threshold for each item. As a result, these algorithms can find frequent itemsets containing rare items and/or frequent items. For example, a user could assign a lower minimum support threshold for the item caviar than for the item bread. A related problem with databases having a skewed item support distribution is that patterns involving both frequent and infrequent items may be spurious as infrequent items may appear with frequent items simply because those latter are frequent. Measures have thus been designed to eliminate such patterns, called *cross-support patterns*[55].

A lot of research has also been carried on finding rare itemsets in databases[13,63,64] instead of frequent itemsets, since frequent itemsets may not always be the most interesting in real-life applications. The problem of discovering rare patterns is quite challenging as there

is generally much more rare patterns than frequent patterns. As a result, various definitions of rare patterns have been proposed, each providing different restrictions on what is a rare pattern to avoid discovering a huge number of rare patterns. For example, the AprioriInverse[62] algorithm finds *perfectly rare itemsets*. A perfectly rare itemset is an itemset that has a support no less than a minimum support threshold and not higher than a maximum support threshold. Furthermore, it is required that all subsets of a perfectly rare itemset also have a support not greater than the maximum support threshold. Another type of rare itemsets is the *minimal rare itemsets*[63,64]. A minimal rare itemset is an itemset that is infrequent according to the minimum support threshold and that all its subsets are frequent. Thus, minimal rare itemsets can be viewed as the itemsets that are almost frequent. Both the perfectly rare itemsets and minimal rare itemsets are subsets of the set of all infrequent itemsets.

Another drawback of traditional FIM algorithms is that they are designed to be applied as batch algorithms, i.e. they are designed to be run only once. This is a problem if FIM algorithms are applied in a dynamic environment. For example, if a transaction database is updated, a user will need to apply a FIM algorithm again to get an updated view of the patterns in the database. This is inefficient because sometimes only small changes are made to a database. In these cases, it would be preferable to not recompute all frequent itemsets from scratch. Various algorithms have been designed to provide updated results to a user when a database changes. There are three main types of algorithms:

- *Incremental mining algorithms* are designed to update the set of frequent itemsets when new transactions are inserted, deleted or modified[65–68] in a transaction database. An example of strategy used by this type of algorithms to avoid recalculating all frequent itemsets from scratch is to keep a buffer of almost-frequent itemsets in memory. Using a buffer, it is unnecessary to perform a database scan to calculate the support of itemsets, when a database is only slightly changed.

- *Stream mining algorithms* are designed to cope with a potentially infinite stream of transactions. They assume that transactions may arrive at a very high speed. Thus, these algorithms are optimized to process transactions as quickly as possible and gen-

erally to calculate an approximate set of frequent itemsets rather than an exact set of frequent itemsets[72]. Some popular algorithms for mining frequent itemsets in streams are estDec[69] and estDec+[70]. These algorithms use a tree structure for maintaining information about frequent itemsets, and also include a recency constraint to discover itemsets that are recently frequent, but that may have been infrequent in the past, and calculate upper-bounds on the approximation error of support calculation for frequent itemsets. Similarly, algorithms have also been designed to find and maintain closed[71] and maximal itemsets[73] in streams.

- *Interactive mining algorithms* propose a different solution to the problem of handling dynamic databases. The idea is that instead of mining and updating a large number of itemsets that may not all be useful, one could mine the itemsets that are needed on-the-fly, when they are needed. For example, the Itemset-Tree[74] and the improved Memory-Efficient Itemset-Tree[75] are tree-based data structures that can be created from a transaction database, and that can be updated incrementally with new transactions. These structures are optimized for quickly answering queries about itemsets on-the-fly such as (1) computing the support of a given itemset, (2) finding all supersets of a given itemset that has a support greater than a given minimum support threshold, and (3) finding all association rules having a given itemset as antecedent. Interactive algorithms can be very efficient. For example, it was reported that the above structure can process more than 10,000 queries in just a few seconds for some datasets, on a standard personal computers[75].

Another important limitation of traditional FIM algorithms is the database format. As previously explained, FIM assumes that the input database only contains binary attributes (items). But in real-life this assumption does not always hold. Thus, several extensions of FIM have been proposed to handle richer database types. Some of the most important ones are the following.

- *Weighted itemset mining* is an extension of frequent itemset mining where weights are associated to each item to indicate their relative importance[76–78]. The goal of weighted itemset mining is to find itemsets that have a minimum weight. A popular variation

of this problem is to mine infrequent weighted itemsets[79].

- *High-utility itemset mining* (HUIM) is an extension of weighted itemset mining where not only weights are considered but also purchase quantities in transactions[20–26]. In traditional FIM, purchase quantities are assumed to be binary, i.e. either an itemset appears in a transaction or not. In HUIM, the number of units bought for each item is indicated in transactions. For example, a transaction could indicate that a customer has bought four breads and two bottles of wine, which is different than a customer having bought one bread and four bottles of wine. In HUIM, weights can be viewed as the unit profit of items (how much profit is generated by each unit sold of a product). The goal of HUIM is to find all itemsets that have a *utility* higher than a given threshold in a database (i.e. itemsets generating a high profit). A major challenge in HUIM is that the utility measure is neither monotone nor anti-monotone[20]. Hence, the utility-measure cannot be directly used to prune the search space. To solve this problem, HUIM algorithms have introduced the concept of upper-bound. For example, the Two-Phase algorithm[20] uses an upper-bound, called the TWU, on the utility of itemsets that is monotone to reduce the search space. A major challenge in HUIM has been to develop tighter upper-bounds on the utility to be able to prune a larger part of the search space, and improve the performance of HUIM algorithms[24–26,67]. One of the current fastest HUIM algorithm is EFIM[25]. Various extensions of the utility-mining problem have also been proposed to consider for example the shelf-time periods of items[80], discount strategies applied in retail stores[81], and also to discover the top-k most profitable itemsets[82,83].

- *Uncertain itemset mining* is another popular extension of FIM, designed to consider uncertainty about the data[84–88]. Uncertainty play a role in several real-life applications since data collected is often imperfect, inaccurate, or may be collected through noisy sensors. Two main models have been proposed for uncertain frequent itemset mining[86]. The first model is the *expected-support model*[85,87,88]. It considers that each item i appearing in a transaction T_q is associated to an expected support value $e(i, T_q)$ representing the certainty that this item appeared in the transaction (a value in the

[0,1] interval). For example, consider a transaction database where items represent symptoms and transactions represent hospital patients. A symptom such as stomach pain may be associated to a patient with an expected-support of 0.75 (representing a probability of 75%) based on medical tests or discussion with the patient. The expected-support of an itemset X in a transaction is defined as the product of the expected-support of its items in the transaction, i.e. $exp(X, T_q) = \prod_{i \in X} e(i, Tq)$. The expected-support of an itemset in a database D is the sum of its expected-support in all transactions where X appears, i.e. $exp(X, D) = \sum_{T_q \in D \wedge X \subseteq T_q} exp(X, Tq)$. The task of uncertain itemset mining in the expected support model is to discover all itemsets that are expected to be frequent. The second model is the *probabilistic itemset model*[85]. It utilizes the same database format, but it considers two thresholds: the minimum support threshold *minsup*, as well as a second threshold called the minimum probability threshold *minprob*. An itemset is considered a *probabilistic frequent itemset* if the calculated probability that it appears in more than *minsup* transactions by considering possible worlds is greater than *minprob*.

- *Fuzzy itemset mining*[89–92] is also a well-studied extension of itemset mining. In fuzzy itemset mining, quantitative values are assigned to each item in transactions and fuzzy membership functions are defined for each attribute (item) to map these values to nominal values. For example, an attribute of a transaction could be the "height" of a person and a corresponding fuzzy membership function could be defined to map a height to nominal values such as short, average or tall. Fuzzy FIM algorithms discover itemsets where each item is associated with a nominal value and a membership percentage. For example, a fuzzy FIM algorithm can discover itemsets such as "height(tall=80%) age(young = 60%)". Some of the most important applications of fuzzy itemset mining are in text mining, including text clustering[91].

Another limitation of frequent itemset mining is that it is traditionally applied to find itemsets in a single set of transactions. However, in real-life, it is often useful to discover patterns that are different or vary greatly in two or more sets of transactions. For example, one may wish to discover patterns that explain the difference between the shopping behavior

of adults and teenagers. For this purpose, algorithms have been designed to mine *emerging patterns*. An emerging pattern is a pattern that is significantly more frequent in a set of transactions than in another[93]. It was shown that emerging patterns are useful for tasks such as classification[94].

Other pattern mining problems related to itemset mining

The previous section has described popular extensions of the problem of FIM to address some of its limitations. This section describes some other important pattern mining problems that are closely related to itemset mining and are solved using similar techniques.

- *Association rule mining*[3,95,96] is the task of discovering rules of the form $X \rightarrow Y$ in a transaction database, where X and Y are itemsets such that $X \cap Y = \emptyset$. The interestingness of a rule is traditionally assessed by the *support* measure $sup(X \rightarrow Y) = sup(X \cup Y)$ and the *confidence* measure $conf(X \rightarrow Y) = sup(X \cup Y)/sup(X)$[3]. The confidence is a very useful measure since it assesses how "strong" an association between some items is. The confidence represents the conditional probability $P(Y|X)$. Thus, for any rule $X \rightarrow Y$, if follows that $conf(X \rightarrow Y)$ is not necessarily equal to $conf(Y \rightarrow X)$. To discover frequent and confident association rules, a user has to specify a minimum support threshold $minsup$ and a minimum confidence threshold $minconf$. An association rule mining algorithm then finds all the valid association rules, i.e. those having a support and confidence respectively no less than these thresholds. For example, for the database of Figure 1 and $minsup = 3$ and $minconf = 1$ (which means 100 % confidence), the valid association rules are $\{a\} \rightarrow \{c\}$, $\{e\} \rightarrow \{b\}$, $\{b\} \rightarrow \{e\}$, $\{c, e\} \rightarrow \{b\}$ and $\{b, c\} \rightarrow \{e\}$, which all have a confidence of 100%, and respectively have a support of 3, 4, 4, 3 and 3 transactions. Using the confidence measure, association rule mining algorithms can discover patterns representing strong associations between items. This addresses a limitation of traditional frequent itemset mining, which is that it may find many frequent itemsets that are weakly correlated.

  For example, $\{a, c\}$ is a frequent itemset for $minsup = 3$. But the rule $\{a\} \rightarrow \{c\}$ has a higher confidence (100%) than the rule $\{c\} \rightarrow \{a\}$ (75%). Thus association rules

can provide a more detailed information about this itemset. Note that more than 20 alternative interestingness measures have been proposed for association mining beside the support and confidence. The interested reader may refer to Lenca et al.[96] for a survey of association rule measures. Association rules are typically generated in two steps by first discovering frequent itemsets using a standard FIM algorithm and then generating the rules using the frequent itemsets[3].

- *Sequential pattern mining*[17,18,97–100] consists of discovering sequences frequently appearing in a set of sequences. The problem is similar to the problem of frequent itemset mining except that the input database is a sequence database, where each sequence is a sequence of transactions. The output of a sequential pattern mining algorithm is a set of *frequent sequential patterns*, that is subsequences that appear in no less than *minsup* sequences in the database, where *minsup* is the minimum threshold specified by the user. Applications of sequential pattern mining include analyzing the genome, analyzing web-click stream, and analyzing alarm data in telecommunications[97]. A variation of the sequential pattern mining problem is to discover *sequential rules*[109,110] of the form $X \rightarrow Y$ indicating that if some items X appear in a sequence it will be followed by some other items Y with a given confidence. A sequential rule $X \rightarrow Y$ can be viewed as an association rule that respect the restriction that X must appear before Y. Sequential rules have been reported as more effective than sequential patterns for some tasks involving prediction[109]. They have been applied in various applications such as e-learning, web page prefetching, anti-pattern detection, alarm sequence analysis and restaurant recommendation[110]. Another variation is to discover *frequent partial orders*[111,112] rather than sequential patterns or rules. Items in a frequent partial orders are only partially ordered sequentially. Thus a frequent partial order can summarize several sequential patterns[111].

- *Episode mining*[101,102] is similar to the problem of sequential pattern mining except that patterns are mined in a single sequence rather than in a set of sequences. The goal is to discover itemsets or *episode rules* of the form $X \rightarrow Y$ that appears many times in a sequence of transactions. An itemset or rule occurs in a sequence if it appears within

a time window set by the user. Episode mining can be used to analyze various types of data such as web-click streams, telecommunication data, sensor readings, sequences of events on an assembly line and network traffic data[101,102].

- *Periodic pattern mining*[103–105] is the problem of discovering patterns in a single sequence of transactions. The goal of periodic pattern is not just to find patterns that regularly appear in a sequence, but that also appear periodically. The time elapsed or number of transactions between two occurrences of an itemset is called the period length. A user typically has to set parameters on the maximum, minimum or average period lengths to discover periodic patterns[104]. For example, in the database of Table 1, the itemset $\{a, c\}$ could be considered as a periodic pattern, since it appears periodically every two transactions. Applications of periodic pattern mining include stock market analysis, market analysis and bioinformatics[103].

- *Sub-graph mining*[106–108] is another problem with many similarities to the problem of frequent itemset mining. The difference is that the goal is to discover *frequent sub-graphs* in a database of graphs rather than frequent itemsets in transactions. The traditional problem of sub-graph mining requires the user to set a minimum support threshold as in FIM. Some of the key challenges in sub-graph mining is that the search space is generally very large and that it is necessary to design strategies to check if different generated graphs are isomorphic (if generated graphs contain the same number of vertices connected in the same manner)[106]. As for frequent itemset mining, several extensions of the problem of sub-graph mining have also been proposed, for example to mine closed and maximal frequent sub-graphs[107]. Applications of sub-graph mining are varied and include for example the analysis of chemical compounds[106,107].

- *Formal concept analysis* (FCA)[113] is a problem that consists of extracting *formal concepts* from a table of objects (transactions) described using binary attributes (items). A formal concept is the equivalent of a closed itemset in FIM, obtained using $minsup = 0$, and annotated with the set of objects (transactions) where it appears. An interesting aspect of FCA is that it organizes formal concepts as a lattice based on the subset relation, which can be visualized. For example, Fig. 3 shows the lattice of formal

concepts extracted from the database of Table 1. FCA has been extensively studied and there exist many variations of the FCA problem[113].

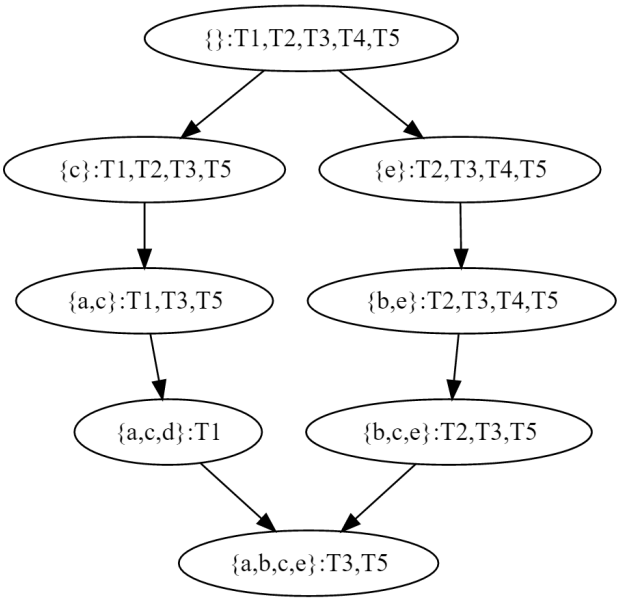


Figure 3: The lattice of formal concepts for the database of Table 1

Research opportunities

Itemset mining and related pattern mining problems have been an active research topic for more than 20 years, and there is still countless opportunities for research in this area. We here provide a classification of the main types of research opportunities in this field:

- *Novel applications.* The first research opportunities is to apply existing pattern mining algorithms in new ways in terms of application domains. Since pattern mining algorithms are quite general, they can be applied in a multitude of domains. In particular, the use of pattern mining methods in emerging research areas such as social network analysis, the Internet of Things, sensor networks provides several novel possibilities in terms of applications.

- *Enhancing the performance of pattern mining algorithms.* Since pattern mining can be quite time-consuming, especially on dense databases, large databases, or databases

containing many long transactions, a lot of research is carried on developing more efficient algorithms. This is an important problem especially for new extensions of the pattern mining problem such as uncertain itemset mining or high-utility itemset mining, which have been less explored. Many opportunities also lies in distributed, GPU, multi-core or parallel algorithm development to increase speed and scalability of the algorithms.

- *Extending pattern mining to consider more complex data.* Another research opportunity is to develop pattern mining algorithms that can be applied on complex types of data. As mentioned in this paper, various extensions have been proposed. But it still remains a problem to handle more complex types of data. Some recent papers have for example focused on mining spatial patterns[114].

- *Extending pattern mining to discover more complex and meaningful types of patterns.* Related to the above opportunity, another important issue to discover more complex types of patterns. Also, another research opportunity is to work on the evaluation of patterns using for example novel interestingness measures[96], because it is also key to ensure that the most interesting or useful patterns are found.

Open-source Software

An issue in the frequent itemset and pattern mining research community is that most researchers do not release the source code of their algorithms, or even their binary files. This has led to the following problems. First, several researchers need to re-implement algorithms proposed by other researchers to compare their algorithm performance thus spending a great amount of time re-implementing algorithms. Second, in these cases, it remains unsure if reimplemented algorithms are as efficient as the original implementations. Besides, even when binary files are released it has been noted in studies such as the one of Goethal[115] that results may vary greatly depending on the compiler used and the machine architecture used for running performance comparison. Third, if algorithm implementations are not released by researchers, fewer people may use them in real applications, as they would need to be

reimplemented.

The main solution to this issue is that researchers release their algorithms as open-source implementations, ideally in a common programming language to facilitate comparison. Currently, the largest collection of open-source implementations of algorithms for pattern mining is by far the *SPMF data mining library*[116] (`http://www.philippe-fournier-viger.com/spmf/`). It offers more than 110 algorithms for mining patterns such as itemsets, sequential patterns, sequential rules, periodic patterns, and association rules. It is a multi-platform library developed in Java and released under the GPL3 license. It is designed to be easily integrated in other Java software programs, and can be run as a standalone software using its command-line or graphical user interface. The other main collection of open-source implementation is the FIMI 2004 competition repository website (`http://fimi.ua.ac.be/`), which provides about twenty C++ implementations of itemset mining algorithms published before 2005. Datasets used for benchmarking itemset and pattern mining algorithms can be found on the SPMF and FIMI websites.

Itemset mining algorithms are also offered as part of some general purpose open-source data mining platform or libraries such as Weka (`http://www.cs.waikato.ac.nz/ml/weka/`), Mahout (`http://mahout.apache.org/`) and Knime (`http://www.knime.org/`), implemented in Java, and R (`https://www.r-project.org/`). These software offer a wide range of data mining techniques. However, they offer a very limited set of algorithms for itemset mining, offering mostly implementations of Apriori, FP-Growth and association rule generation. Some specialized platforms offering itemset mining algorithms are Coron (`http://coron.loria.fr/`) and LUCS-KDD (`http://`),developed in Java, and Illimine (`http://illimine.cs.uiuc.edu/`),developed in C++. However, the source code of Coron is not public, Illimine provides the source code of only one of its pattern mining algorithms and LUCS-KDD source code cannot be used for commercial purposes.

Related to the importance of sharing implementations of algorithms in the research community, another issue is that researchers in the field of pattern mining do not always compare the performance of new algorithms with the previous best algorithms because of the lack of public implementations or other reasons. To illustrate this problem, consider the task of subgraph mining. Various algorithms have been proposed for this task. Among them, the GAS-

TON , FFSM, SPIN algorithms have been shown to outperform GSPAN[107]. Furthermore, SPIN and MARGIN were shown to outperform FFSM[107]. And recently, FPGraphMiner[117] was shown to outperform MARGIN and FFSM. However, to our knowledge no algorithm has been compared to GASTON, even though it is one of the first proposed algorithms for frequent sub-graph mining. And SPIN, GASTON and FPGraphMiner have also never been compared with each other. Moreover, can we really make the transitive inference that an algorithm like FPGraphMiner is faster than GSPAN, because it is faster than MARGIN and FFSM? Thus, in this type of cases, it remains uncertain, which algorithm is the most efficient.

CONCLUSIONS

Itemset mining is an active field of research having numerous applications. This paper has presented the problem of frequent itemset mining, discussed the main techniques for exploring the search space of itemsets, employed by itemset mining algorithms. Then, the paper has discussed several extensions of the basic frequent itemset mining problem to overcome some of its limitations, for example, to handle dynamic databases, uncertain data and the use of various constraints. The paper has also briefly discussed pattern mining problems that are closely related to the problem of itemset mining such as sequential pattern mining, sequential rule mining, association rule mining, periodic pattern mining and episode mining. Lastly, the paper has discussed research opportunities and open-source implementations of pattern mining algorithms.

Acknowledgement. This work is supported by the Youth 1000 Talent funding from the National Science Fundation of China, and an initiating fund from the Harbin Institute of Technology.

References

1. Aggarwal, CC. *Data mining: the textbook*. Heidelberg:Springer; 2015.

2. Han, J, Pei, J, Kamber, M. *Data mining: concepts and techniques*. Amsterdam:Elsevier; 2011.

3. Agrawal, R, Srikant, R. Fast algorithms for mining association rules. In: *Proc. 20th int. conf. very large data bases, VLDB 1994*, Santiago de Chile, Chile, 12-15 September, 1994: 487–499).

4. Pei, J, Han, J Lakshmanan, LV. Pushing convertible constraints in frequent itemset mining. *Data Mining and Knowledge Discovery*, 2004, 8(3):227-252.

5. Naulaerts, S, Meysman, P, Bittremieux, W, Vu, TN, Berghe, WV, Goethals, B, Laukens, K. A primer to frequent itemset mining for bioinformatics. *Briefings in bioinformatics*, 2015, 16(2), 216–231.

6. Fernando, B, Elisa F, Tinne T. Effective use of frequent itemset mining for image classification. In: *European Conference on Computer Vision*. Florence, Italy, 7-13 October, 2012: 214–227.

7. Glatz, E, Mavromatidis, S, Ager, B, Dimitropoulos, X. Visualizing big network traffic data using frequent pattern mining and hypergraphs. *Computing*, 2014, 96(1):27–38.

8. Brauckhoff, D, Dimitropoulos, X, Wagner, A, Salamatian, K. Anomaly extraction in backbone networks using association rules. *IEEE/ACM Transactions on Networking*, 2012, 20(6):1788–1799

9. Mukherjee, A, Liu, B, Glance, N. Spotting fake reviewer groups in consumer reviews. In: *Proc. 21st international conference on World Wide Web*. Lyon, France, 16-20 April, 2012: 191–200.

10. Liu, Y, Zhao, Y, Chen, L, Pei, J, Han, J. Mining frequent trajectory patterns for activity monitoring using radio frequency tag arrays. *IEEE Transactions on Parallel and Distributed Systems*, 2012, 23(11):2138–2149.

11. Duan, Y, Fu, X, Luo, B, Wang, Z, Shi, J, Du, X. Detective: Automatically identify and analyze malware processes in forensic scenarios via DLLs. In: *Proc. 2015 IEEE*

International Conference on Communications, London, United Kingdom, 8-12 June, 2015: 5691–5696.

12. Mwamikazi, E, Fournier-Viger, P, Moghrabi, C, Baudouin, R. A Dynamic Questionnaire to Further Reduce Questions in Learning Style Assessment. In: *Proc. 10th Int. Conf. Artificial Intelligence Applications and Innovations*, Rhodes, Greece, 19-21 September, 2014: 224–235.

13. Koh, YS, Ravana, SR. Unsupervised Rare Pattern Mining: A Survey. *ACM Transactions on Knowledge Discovery from Data*, 2016, 10(4): article no. 45.

14. Fournier-Viger, P, Lin, JCW, Dinh, T, Le, HB. Mining Correlated High-Utility Itemsets using the Bond Measure. In: *Proc. Intern. Conf. Hybrid Artificial Intelligence Systems*. Seville, Spain, 18-20 April, 2016:53–65).

15. Barsky, M, Kim, S, Weninger, T, Han, J. Mining flipping correlations from large datasets with taxonomies. *VLDB Endowment*, 2011, 5(4):370–381.

16. Soulet, A, Raissi, C, Plantevit, M, Cremilleux, B. Mining dominant patterns in the sky. In: *Proc. 11th IEEE Int. Conf. on Data Mining*, Vancouver, Canada, 11-14 December, 2011:655–664.

17. Mabroukeh NR, Ezeife CI. A taxonomy of sequential pattern mining algorithms. *ACM Computing Surveys*. 2010, 43(1):3.

18. Fournier-Viger, P, Gomariz, A, Campos, M, Thomas, R. Fast Vertical Mining of Sequential Patterns Using Co-occurrence Information. In: *Proc. 18th Pacific-Asia Conf. Knowledge Discovery and Data Mining*. Tainan, Taiwan, 13-16 May, 2014:40–52.

19. Yan, X, Han, J. 2002. gspan: Graph-based substructure pattern mining. In: *Proc. 2002 Intern. Conf. Data Mining*. Maebashi City, Japan, 9-12 December, 2002:721–724.

20. Liu, Y., Liao, W.K. and Choudhary, AN. A two-phase algorithm for fast discovery of high utility itemsets. In: *Pacific-Asia Conference on Knowledge Discovery and Data Mining*, Hanoi, Vietnam, 18-20 May, 2005:689–695.

21. Lin, CW, Hong, TP, Lu, WH. An effective tree structure for mining high utility itemsets. *Expert Systems with Applications*. 2011,30;38(6):7419–24.

22. Lin, YC, Wu, CW, Tseng, VS. Mining high utility itemsets in big data. In: *Proc. Pacific-Asia Conference on Knowledge Discovery and Data Mining* 2015. Ho Chi Minh City, Vietnam, 19-22 May, 2015:649–661.

23. Liu, M, Qu,. J. Mining high utility itemsets without candidate generation. In: *Proc. 21st ACM Intern. Conf. Information and knowledge management*. Maui, USA, 29 October - 2 November, 2012:55–64.

24. Fournier-Viger P, Wu CW, Zida S, Tseng VS. FHM: Faster high-utility itemset mining using estimated utility co-occurrence pruning. In: *Proc. Inter. Symp. Methodologies for Intelligent Systems*, Roskilde, Denmark, 25-27 June, 2014:83–92.

25. Zida, S., Fournier-Viger, P, Lin, JC.W, Wu, CW, Tseng, VS. EFIM: A Highly Efficient Algorithm for High-Utility Itemset Mining. In: *Proc. 14th Mexican Intern. Conf. Artificial Intelligence*, Cuernavaca, Mexico, 25-31 October, 2015:530–546.

26. Yun, U, Ryang, H, Ryu, KH. High utility itemset mining with techniques for reducing overestimated utilities and pruning candidates. *Expert Syst. Appl.* 2014, 41(8):3861-3878.

27. Han, J, Pei, J, Ying, Y, Mao, R. Mining frequent patterns without candidate generation: a frequent-pattern tree approach. *Data Min. Knowl. Discov.*, 2004, 8(1):53–87.

28. Zaki, MJ. Scalable Algorithms for Association Mining. *IEEE Trans. Knowl. Data Eng.*, 2000, 12(3):372–390, 2000.

29. Pei, J, Han, J, Lu, H, Nishio, S, Tang, S, Yang, D,H-mine: Hyper-structure mining of frequent patterns in large databases. In: *Proc. 2001 IEEE Intern. Conf. Data Mining*, San Jose, USA, 29 November - 2 December, 2001:441–448.

30. Uno, T, Kiyomi, M, Arimura, H. LCM ver. 2: Efficient mining algorithms for frequent/closed/maximal itemsets. *Proc. ICDM'04 Workshop on Frequent Itemset Mining Implementations*, CEUR, 2004.

31. Hegland M. The apriori algorithma tutorial. *Mathematics and computation in imaging science and information processing*. 2005;11:209-62.

32. Zaki, MJ, Gouda, K. Fast vertical mining using diffsets. In: *Proc. 9th ACM SIGKDD Intern. Conf. Knowledge Discovery and Data Mining*, Washington DC, USA, 24 - 27 August, 2003:326-335.

33. Lucchese, C., Orlando, S., Perego, R. Fast and Memory Efficient Mining of Frequent Closed Itemsets. *IEEE Trans. Knowl. Data Eng.,* 2006, 18(1):21–36.

34. Chen J, Xiao K. BISC: A bitmap itemset support counting approach for efficient frequent itemset mining. *ACM Transactions on Knowledge Discovery from Data*. 2010, 1;4(3):12.

35. Deng ZH, Lv SL. PrePost+: An efficient N-lists-based algorithm for mining frequent itemsets via ChildrenParent Equivalence pruning. *Expert Systems with Applications*. 2015, 1;42(13):5424–3

36. Pyun G, Yun U, Ryu KH. Efficient frequent pattern mining based on linear prefix tree. *Knowledge-Based Systems*. 2014, 31;55:125-39.

37. Vo, B, Le, T, Coenen, F. Hong, TP. Mining frequent itemsets using the N-list and subsume concepts. *International Journal of Machine Learning and Cybernetics*. 2016, 7(2):253–265.

38. Zhang F, Zhang Y, Bakos JD. Accelerating frequent itemset mining on graphics processing units. *The Journal of Supercomputing*. 2013, 1;66(1):94-117.

39. Schlegel B, Karnagel T, Kiefer T, Lehner W. Scalable frequent itemset mining on many-core processors. In: *Proc. 9th Intern. Workshop Data Management on New Hardware*, New York, USA, 24 June, 2013:paper 3.

40. Moens S, Aksehirli E, Goethals B. Frequent itemset mining for big data. In: *2013 IEEE International Conference on Big Data*, Santa Clara, USA, 6-9 October, 2013:111–118.

41. Qiu H, Gu R, Yuan C, Huang Y. Yafim: a parallel frequent itemset mining algorithm with spark. In: *Proc. 2014 IEEE International Parallel and Distributed Processing Symposium Workshops*, Phoenix, USA, 19-23 May, 2014:1664–1671.

42. Zaki, MJ, Hsiao, CJ, CHARM: An efficient algorithm for closed itemset mining. In: *Proc. 12th SIAM Intern. Conf. Data Mining*, Anaheim, USA, 26-28 April, 2012:457–473.

43. Pasquier N, Bastide Y, Taouil R, Lakhal L. Discovering frequent closed itemsets for association rules. In: *Proc. Intern. Conf. Database Theory*, Jerusalem, Israel, 10-12 January, 1999:398–416.

44. Aliberti, G, Colantonio, A, Di Pietro, R, Mariani, R. EXPEDITE: EXPress closED ITemset Enumeration. *Expert Systems with Applications*, 2015, 42(8):3933–3944.

45. Vo B, Hong TP, Le B. DBV-Miner: A Dynamic Bit-Vector approach for fast mining frequent closed itemsets. *Expert Systems with Applications*. 2012, 39(8):7196–206.

46. Soulet A, Rioult F. Efficiently depth-first minimal pattern mining. In: *Proc. 18th Pacific-Asia Conf. Knowledge Discovery and Data Mining*. Tainan, Taiwan, 13-16 May, 2014:28-39.

47. Fournier-Viger P, Wu CW, Tseng VS. Novel concise representations of high utility itemsets using generator patterns. In: *Proc. Intern. Conf. International Conference on Advanced Data Mining and Applications*, Guilin, China, 19-21 December, 2014:30–43.

48. Szathmary L, Valtchev P, Napoli A, Godin R, Boc A, Makarenkov V. A fast compound algorithm for mining generators, closed itemsets, and computing links between equivalence classes. *Annals of Mathematics and Artificial Intelligence*. 2014, 1;70(1-2):81-105.

49. Barron, A, Rissanen, J, Yu, B. The minimum description length principle in coding and modeling. *IEEE Transactions on Information Theory*. 1998, 44(6):2743-60.

50. Antonie, L, Zaiane, OR, Holte, RC. Redundancy reduction: does it help associative classifiers? In: *Proc. 31st Annual ACM Symposium on Applied Computing*, Pisa, Italy, 4-8 April, 2016:867–874

51. Srikant, R, Vu, Q, Agrawal, R, Mining Association Rules with Item Constraints. In: *Proc. 3rd Intern. Conf. Knowledge Discovery and Data Mining*, Newport Beach, USA, 14-17 August, 1997:67–73.

52. Tang L, Zhang L, Luo P, Wang M. Incorporating occupancy into frequent pattern mining for high quality pattern recommendation. In: *Proc. 21st ACM Intern. Conf. Information and knowledge management*, In: *Proc. 21st ACM Intern. Conf. Information and knowledge management*. Maui, USA, 29 October - 2 November, 2012:75-84.

53. Geng L, Hamilton HJ. Interestingness measures for data mining: A survey. *ACM Computing Surveys*. 2006, 30;38(3):9.

54. Omiecinski, E. Alternative Interest Measures for Mining Associations in Databases. *IEEE Transactions on Knowledge Discovery and Data Engineering*. 2003, 15(1):57-69.

55. Xiong H, Tan PN, Kumar V. Mining strong affinity association patterns in data sets with skewed support distribution. In: Proc. 2003 IEEE Intern. Conf. Data Mining. Melbourne, USA, 19-22 December, 2003:387–394.

56. Pei, J, Han, J, Lakshmanan, LV. Mining frequent itemsets with convertible constraints. In: *Proc. 17th Intern. Conf. Data Engineering,* Heidelberg, Germany 2-6 April, 2001:433-442.

57. Pei, J, Han, J, Lakshmanan, LV. 2004. Pushing convertible constraints in frequent itemset mining. *Data Mining and Knowledge Discovery*, 8(3):227-252.

58. Bonchi F, Lucchese C. Pushing tougher constraints in frequent pattern mining. In: *Pacific-Asia Conference on Knowledge Discovery and Data Mining*, Hanoi, Vietnam, 18-20 May, 2005:114–124).

59. Liu, B., Hsu, W., Ma, Y. Mining Association Rules with Multiple Minimum Supports. In: *Proc. ACM SIGKDD Intern. Conf. Knowledge Discovery and Data Mining*, San Diego, USA, 15-18 August, 1999:337-341.

60. Hu, YH, Chen, YL. Mining association rules with multiple minimum supports: a new mining algorithm and a support tuning mechanism. *Decision Support Systems*, 2006, 42(1):1–24.

61. Kiran, R. U, Reddy, PK. Novel techniques to reduce search space in multiple minimum supports-based frequent pattern mining algorithms. In: *Proc. 14th Intern. Conf. Extending Database Technology*, Uppsala, Sweden, 21-24 March, 2011:11–20.

62. Koh, YS, Rountree, N. Finding Sporadic Rules Using Apriori-Inverse. In: *Proc. 9th Pacific-Asia Conference, PAKDD 2005*, Hanoi, Vietnam, 18-20 May, 2005:97–106.

63. Szathmary, L, Napoli, A, Valtchev, P. Towards Rare Itemset Mining. In: *Proc. 19th IEEE Intern. Conf. Tools with Artificial Intelligence*, Patras, Greece, 29-31 October, 2007:305–312.

64. Szathmary, L, Valtchev, P, Napoli, A, Godin, R. Efficient Vertical Mining of Minimal Rare Itemsets. In: *Proc. 9th Intern. Conf. Concept Lattices and Their Applications*, Fuengirola, Spain, 11-14 October, 2012: 269–280.

65. Koh JL, Shieh SF. An efficient approach for maintaining association rules based on adjusting FP-tree structures. In: *Proc. 9th Intern. Conf. Database Systems for Advanced Applications*, Jeju Island, Korea, 17-19 March, 2004:417–424.

66. Leung CK, Khan QI, Li Z, Hoque T, CanTree: a canonical-order tree for incremental frequent-pattern mining. *Knowledge and Information Systems*. 2007, 1;11(3):287–311.

67. Lin CW, Hong TP, Lu WH. The Pre-FUFP algorithm for incremental mining. *Expert Systems with Applications*. 2009, 31;36(5):9498–505.

68. Nath B, Bhattacharyya DK, Ghosh A. Incremental association rule mining: a survey. *Wiley Interdisciplinary Reviews: Data Mining and Knowledge Discovery*, 2013, 1;3(3):157–69.

69. Chang, JH, Lee, WS. Finding recent frequent itemsets adaptively over online data streams. In: *Proc. 9th ACM SIGKDD Intern. Conf. Knowledge Discovery and Data Mining*, Washington DC, USA, 24 - 27 August, 2003:487-492

70. Shin, SJ, Lee, DS, Lee, WS. CP-tree: An adaptive synopsis structure for compressing frequent itemsets over online data streams, *Information Sciences*, 2014, 278;10:559-576

71. Chi Y, Wang H, Philip SY, Muntz RR. Catch the moment: maintaining closed frequent itemsets over a data stream sliding window. *Knowledge and Information Systems*. 2006, 10(3):265–94.

72. Calders T, Dexters N, Gillis JJ, Goethals B. Mining frequent itemsets in a stream. *Information Systems*, 2014, 31;39:233–55.

73. Farzanyar Z, Kangavari M, Cercone N. Max-FISM: Mining (recently) maximal frequent itemsets over data streams using the sliding window model. *Computers and Mathematics with Applications*. 2012, 30;64(6):1706–18.

74. Kubat, M, Hafez, A, Vijay Raghavan, V, Lekkala, JR, Chen, WK. Itemset Trees for Targeted Association Querying. *IEEE Trans. Knowl. Data Eng.,* 2003, 15(6):1522–1534.

75. Fournier-Viger, P., Mwamikazi, E., Gueniche, T., Faghihi, U. (2013). Memory Efficient Itemset Tree for Targeted Association Rule Mining. *Proc. 9th Intern. Conf. Advanced Data Mining and Applications*, Hangzhou, China, 14-16 December, 2013:95–106.

76. Torres-Verdn C, Chiu KY, Vasudeva Murthy AS. WFIM: weighted frequent itemset mining with a weight range and a minimum weight. In: *Proc. 2005 SIAM Intern. Conf. Data Mining*, Newport Beach, USA, 21-23 April, 2005:636–640.

77. Yun, U. Efficient mining of weighted interesting patterns with a strong weight and/or support affinity. *Information Sciences*, 2007, 177(17): 3477-3499.

78. Yun, U. On pushing weight constraints deeply into frequent itemset mining. *Intelligent Data Analysis*, 2009, 13(2):359-383.

79. Cagliero, L, Garza, P, Infrequent weighted itemset mining using frequent pattern growth. *IEEE transactions on knowledge and data engineering*, 2014, 26(4):903-915.

80. Fournier-Viger, P, Zida, S. FOSHU: Faster On-Shelf High Utility Itemset Mining with or without negative unit profit. *Proc. 30th Symposium on Applied Computing*. Salamanca, Spain, 13-17 April, 2015:857–864.

81. Lin, JC. W, Gan, W, Fournier-Viger, P, Hong, TP, Tseng, VS. Fast Algorithms for Mining High-Utility Itemsets with Various Discount Strategies. *Advanced Engineering Informatics*, 2016.

82. Tseng, V, Wu, C, Fournier-Viger, P, Yu, PS. Efficient Algorithms for Mining Top-K High Utility Itemsets. *IEEE Transactions on Knowledge and Data Engineering*, 2016, 28(1):54–67.

83. Duong, QH., Liao, B, Fournier-Viger, P, Dam, TL. An efficient algorithm for mining the top-k high utility itemsets, using novel threshold raising and pruning strategies. *Knowledge-Based Systems*, 2016.

84. Bernecker, T, Kriegel, HP, Renz, M, Verhein, F, Zuefle, A. Probabilistic frequent itemset mining in uncertain databases. In: *Proc. 15th ACM SIGKDD Intern. Conf. Knowledge discovery and data mining*, Paris, France, June 28 - July 1, 2009:119–128.

85. Chui, CK, Kao, B, Hung, E. Mining frequent itemsets from uncertain data. In: *Pacific-Asia Conference on Knowledge Discovery and Data Mining*, Nanjing, China, 22-25 May, 2007:47–58.

86. Tong, Y, Chen, L, Cheng, Y, Yu, PS. Mining frequent itemsets over uncertain databases. *VLDB Endowment*, 2012, 5(11):1650–1661.

87. Leung, CKS, MacKinnon, RK. (2014, September). BLIMP: a compact tree structure for uncertain frequent pattern mining. In: *Proc. Intern. Conf. Data Warehousing and Knowledge Discovery*, Munich, Germany, 2-4 September, 2014:115–123.

88. Lin, JCW, Gan, W, Fournier-Viger, P, Hong, TP, Tseng, VS. Weighted frequent itemset mining over uncertain databases. *Applied Intelligence*, 2015, 44(1):232–250.

89. Chen, CH, Li, AF, Lee, YC, Actionable high-coherent-utility fuzzy itemset mining. *Soft Computing*, 2014, 18(12):2413–2424.

90. Hong TP, Kuo CS, Wang SL. A fuzzy AprioriTid mining algorithm with reduced computational time. *Applied Soft Computing*. 2004, 31;5(1):1–10.

91. Chen CL, Tseng FS, Liang T. Mining fuzzy frequent itemsets for hierarchical document clustering. *Information processing and management*. 201, 31;46(2):193–211.

92. Lin, JCW, Tin, L, Fournier-Viger, P, Hong, TP. A fast Algorithm for mining fuzzy frequent itemsets. *Journal of Intelligent and Fuzzy Systems*, 2015, 9(6):2373–2379.

93. Dong G, Li J. Efficient mining of emerging patterns: Discovering trends and differences. In: *Proc. ACM SIGKDD Intern. Conf. Knowledge Discovery and Data Mining*, San Diego, USA, 15-18 August, 1999:43-52.

94. Garca-Borroto M, Martinez-Trinidad JF, Carrasco-Ochoa, JA. A survey of emerging patterns for supervised classification. *Artificial Intelligence Review*. 2014, 42(4):705-21.

95. Fournier-Viger, P, Wu, CW, Tseng, VS Mining Top-K Association Rules. In: *Proc. 25th Canadian Conf. on Artificial Intelligence*, Toronto, Canada, 28-30 May, 2012: 61-73.

96. Lenca P, Vaillant B, Meyer P, Lallich S. Association rule interestingness measures: Experimental and theoretical studies. In: *Proc. of the Quality Measures in Data Mining Workshop*. 2007:51-76.

97. Mooney CH, Roddick JF. Sequential pattern mining–approaches and algorithms. *ACM Computing Surveys*. 2013, 45(2):1–19.

98. Ayres, J, Flannick, J, Gehrke, J, Yiu, T. Sequential pattern mining using a bitmap representation. In: *Proc. 8th ACM SIGKDD Intern. conf. knowledge discovery and data mining*, Edmonton, Alberta, 23-26 July, 2002: 429–435.

99. Pei J, Han J, Mortazavi-Asl B, Wang J, Pinto H, Chen Q, Dayal U, Hsu MC. Mining sequential patterns by pattern-growth: The prefixspan approach. *IEEE Transactions on knowledge and data engineering*. 2004, 16(11):1424–40.

100. Gouda K, Hassaan M, Zaki MJ. Prism: An effective approach for frequent sequence mining via prime-block encoding. *Journal of Computer and System Sciences*. 2010, 28;76(1):88–102.

101. Mannila H, Toivonen H, Verkamo AI. Discovery of frequent episodes in event sequences. *Data mining and knowledge discovery*. 1997, 1;1(3):259-89.

102. Zimmermann A. Understanding episode mining techniques: Benchmarking on diverse, realistic, artificial data. *Intelligent Data Analysis*. 2014, 1;18(5):761-91.

103. Fournier-Viger, P, Lin, CW, Duong, QH., Dam, TL. PHM: Mining Periodic High-Utility Itemsets. In: *Proc. 16th Industrial Conference on Data Mining*. New York, USA, 13-17 July, 2016:64–79.

104. Tanbeer SK, Ahmed CF, Jeong BS, Lee YK. Discovering periodic-frequent patterns in transactional databases. In: *Pacific-Asia Conference on Knowledge Discovery and Data Mining*, Bangkok, Thailand, 27-30 April, 2009:242–253.

105. Kiran, U, Venkatesh, JN, Fournier-Viger, P, Toyoda, M, Reddy, PK, Kitsuregawa, M. Discovering Periodic Patterns in Non-Uniform Temporal Databases. *Proc. 21th Pacific-Asia Conference on Knowledge Discovery and Data Mining*, Jeju, Korea, 23-27 May, 2017.

106. Yan, X, Han, J. gSpan: Graph-Based Substructure Pattern Mining. In: *Proc. 2002 IEEE Intern. Conf. Data Mining*, Maebashi City, Japan, 9-12 December, 2002:721-724

107. Jiang C, Coenen F, Zito M. A survey of frequent subgraph mining algorithms. *The Knowledge Engineering Review*. 2013, 28(1):75–105.

108. Bhuiyan MA, Al Hasan M. An iterative MapReduce based frequent subgraph mining algorithm. *IEEE Transactions on Knowledge and Data Engineering*. 2015, 1;27(3):608-20.

109. Fournier-Viger, P, Wu, CW, Tseng, VS, Cao, L, Nkambou, R, Mining Partially-Ordered Sequential Rules Common to Multiple Sequences. *IEEE Transactions on Knowledge and Data Engineering*, 2015, 27(8):2203-2216.

110. Fournier-Viger P, Gueniche T, Zida S, Tseng VS. ERMiner: sequential rule mining using equivalence classes. In: *Proc. 13th Intern. Symp. Intelligent Data Analysis*, Leuven, Belgium, October 30 - November 1, 2014:108-119.

111. Fabregue, M, Braud, A, Bringay, S, Le Ber, F, Teisseire, M, Mining closed partially ordered patterns, a new optimized algorithm. *Knowledge-Based Systems*, 2015, 79:68-79.

112. Pei, J, Wang, H, Liu, J, Wang, K, Wang, J, Yu, PS. Discovering frequent closed partial orders from strings. *IEEE Transactions on Knowledge and Data Engineering*, 2006, 18(11):1467-1481.

113. Poelmans, J, Kuznetsov, SE, Ignatov, DI, Dedene, G, Formal Concept Analysis in knowledge processing: A survey on models and techniques. *Expert Systems with Applications*, 2013, 40(16):6601-6623

114. Sengstock, C, Gertz, M. Spatial Itemset Mining: A Framework to Explore Itemsets in Geographic Space. In: *Proc. East European Conference on Advances in Databases and Information Systems*, Genoa, Italy, 1-4 September, 2013:148-161

115. Goethals B. *Survey on frequent pattern mining*. Univ. of Helsinki. 2003.

116. Fournier-Viger, P, Gomariz, A, Gueniche, T, Soltani, A, Wu, CW, Tseng, VS. SPMF: a Java Open-Source Pattern Mining Library, *Journal of Machine Learning Research*, 2014, 15:3389–3393.

117. Vijayalakshmi, R, Nadarajan, R, Roddick, JF, Thilaga, M, Nirmal,a P. FP-GraphMiner-A Fast Frequent Pattern Mining Algorithm for Network Graphs. *J. Graph Algorithms Appl.*. 2011, 15(6):753-76.

View publication stats